

Relatório do Sistema de Gerenciamento de Biblioteca com TADs – Trabalho Prático de Estrutura de Dados



**UNIVERSIDADE
ESTADUAL DO CEARÁ**

Data: 16/06/2026

Curso: Ciência da computação

Disciplina: Estrutura de dados

Professor: Bonfim Amaro Junior

Equipe:

- Gustavo de Melo Timbó
- João Lucas Simões Rocha da Silveira
- José Moisés da Silva Lima

1. Introdução

Este trabalho apresenta o desenvolvimento de um Sistema de Gerenciamento de Biblioteca em linguagem C, com foco no controle do acervo de livros, das reservas e do histórico de empréstimos. O problema proposto exige o cadastro e a consulta de livros identificados por um código único, além da listagem ordenada, do empréstimo e da devolução de exemplares. Para atender a essas necessidades, a solução foi construída com base em Tipos Abstratos de Dados e nas estruturas árvore binária de busca, fila e lista encadeada.

A escolha dessas estruturas de dados é relevante porque a organização correta das informações influencia diretamente na eficiência do sistema. A árvore binária de busca permite armazenar os livros de forma ordenada pelo código, facilitando buscas e percursos; a fila controla os usuários que aguardam reserva quando não há exemplares disponíveis; e a lista encadeada registra os empréstimos realizados, mantendo um histórico organizado das operações. Assim, o sistema não apenas resolve um problema prático de gerenciamento, como também evidencia a aplicação integrada de estruturas fundamentais da disciplina de Estrutura de Dados.

2. Descrição dos TADs

- TAD Livro

O TAD Livro representa cada obra cadastrada no sistema da biblioteca. Ele é composto pelos campos `codigo`, `titulo`, `autor`, `ano`, `quantidadeTotal` e `quantidadeDisponivel`, que armazenam, respectivamente, o identificador único do livro, seu título, autor, ano de publicação e o controle de exemplares totais e disponíveis. Esse TAD é a base do sistema, pois é a partir dele que os livros são armazenados na árvore binária de busca e manipulados nas operações de cadastro, busca, empréstimo e devolução.

Além da estrutura de dados, o TAD Livro possui funções próprias para criar um novo livro, exibir suas informações, obter o código identificador, consultar a quantidade disponível e atualizar o número de exemplares por meio das operações de empréstimo e devolução.

- TAD Reserva

O TAD Reserva representa um usuário que aguardou um livro sem exemplares disponíveis. Ele armazena dois dados principais: `nomeUsuario` e `codigoLivro`. Esses campos são suficientes para identificar quem está na fila de espera e qual livro foi solicitado. No sistema, esse TAD é utilizado pela fila de reservas, que organiza os pedidos na ordem em que foram feitos.

- TAD Empréstimo

O TAD Empréstimo representa o registro de um empréstimo concluído com sucesso. Ele contém os campos `nomeUsuario`, `codigoLivro` e `tituloLivro`.

Diferentemente da reserva, esse tipo não serve para indicar espera, mas sim para registrar uma operação já realizada.

No sistema, cada empréstimo aprovado é inserido em uma lista encadeada de histórico, permitindo consultar posteriormente quais livros foram emprestados e para quais usuários. Isso torna o TAD Empréstimo importante para o controle das movimentações da biblioteca, pois ele documenta as operações realizadas e dá suporte à funcionalidade de exibição do histórico.

3. Descrição da Árvore Binária de Busca

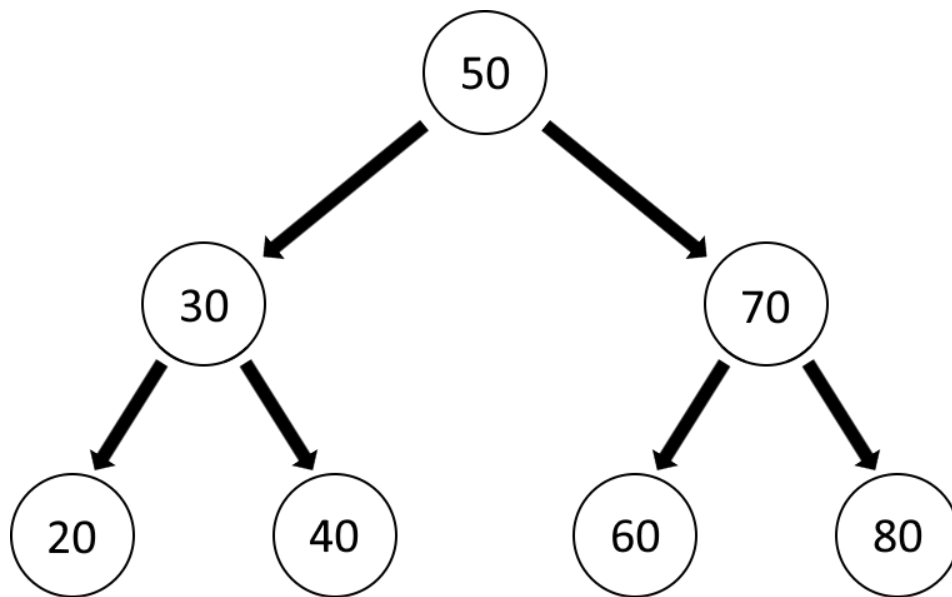
A estrutura principal de armazenamento do sistema é uma árvore binária de busca, utilizada para organizar o acervo de livros da biblioteca a partir do código identificador de cada livro. No projeto, cada nó da árvore guarda um ponteiro para um objeto do tipo Livro, permitindo que as informações do livro fiquem separadas da estrutura de organização, o que torna o sistema mais modular e facilita a integração com o restante da aplicação.

O critério de ordenação adotado é simples: todo livro com código menor que o código do nó atual é inserido na subárvore da esquerda, enquanto todo livro com código maior segue para a subárvore da direita. Dessa forma, a árvore mantém a propriedade fundamental de uma árvore binária de busca, na qual os elementos ficam organizados de modo que a busca por código possa ser feita de forma eficiente. No caso deste sistema, não são aceitos dois livros com o mesmo código, o que evita duplicidade de registros e preserva a integridade do acervo.

As operações implementadas para a árvore incluem a criação da estrutura, a inserção de livros, a busca por código, a listagem em ordem crescente, a listagem em pré-ordem, a listagem em pós-ordem, a contagem de livros e o cálculo da altura da árvore. Além dessas operações principais, o código também prevê rotinas de salvamento, carregamento e liberação de memória, que são úteis para persistência dos dados e encerramento correto do programa.

A inserção de um novo livro ocorre de forma recursiva. Primeiro, o sistema verifica se a árvore está vazia; se estiver, o novo livro passa a ser a raiz. Caso contrário, o código do livro é comparado com o código do nó atual até que seja encontrado o ponto correto da inserção. Por exemplo, se a árvore já contiver os livros de código 50 e 30, e for inserido um livro de código 40, ele será comparado com 50 e seguirá para a esquerda; em seguida, será comparado com 30 e, por ser maior, será inserido à direita de 30. Após isso, também é feito um balanceamento da árvore para evitar gerar árvores muito extensas. Todo esse processo mantém a organização da árvore sem necessidade de reordenar todos os elementos.

Um exemplo de percurso em ordem ajuda a mostrar a principal vantagem da árvore binária de busca. Considere a seguinte árvore:



Ao aplicar o percurso em ordem, a visita acontece na sequência esquerda → nó → direita, resultando na saída: 20, 30, 40, 50, 60, 70, 80. No percurso em pré-ordem, a visita ocorre na sequência nó → esquerda → direita, produzindo a saída: 50, 30, 20, 40, 70, 60, 80. Já no percurso em pós-ordem, a visita segue a sequência esquerda → direita → nó, obtendo a saída: 20, 40, 30, 60, 80, 70, 50.

Outra funcionalidade implementada é a busca de livros por código. A operação utiliza a propriedade de ordenação da árvore para localizar um elemento de forma eficiente. A partir da raiz, o código procurado é comparado com o código armazenado no nó atual. Se os valores forem iguais, o livro é encontrado; se o código procurado for menor, a busca continua na subárvore esquerda; caso seja maior, a busca prossegue na subárvore direita. Esse processo é repetido recursivamente até que o livro seja localizado ou que se chegue a um ponteiro nulo, indicando que o código não está cadastrado.

O sistema também possui uma função para contagem de livros cadastrados. Essa operação percorre toda a árvore recursivamente e soma a quantidade de nós existentes. Cada nó corresponde a um livro armazenado no acervo, de modo que o resultado obtido representa o número total de livros cadastrados no sistema.

Outra operação importante é o cálculo da altura da árvore. A altura corresponde ao maior número de arestas entre a raiz e uma folha. Para determinar esse valor, o algoritmo calcula recursivamente as alturas das subárvores esquerda e direita de cada nó e retorna a maior delas acrescida de uma unidade. Essa informação permite avaliar o formato da árvore e influencia diretamente o desempenho das operações de busca e inserção.

Por fim, a implementação inclui funções auxiliares para salvamento e carregamento dos dados em arquivo, permitindo que os livros cadastrados sejam preservados entre diferentes execuções do programa. Também foi desenvolvida uma rotina de liberação de memória, responsável por percorrer toda a árvore e desalocar corretamente os nós e os livros armazenados, evitando vazamentos de memória e garantindo o encerramento adequado da aplicação.

4. Descrição da Fila

A estrutura de uma fila foi implementada para controlar os usuários que aguardam um livro quando não há exemplares disponíveis no acervo. No sistema, cada reserva armazena o nome do usuário e o código do livro solicitado, permitindo registrar de forma simples qual pessoa está esperando por qual obra.

A política adotada pela fila é a FIFO (First In, First Out), ou seja, o primeiro usuário que entra na fila é o primeiro a ser atendido. Esse comportamento é o mais adequado para reservas, pois respeita a ordem de chegada dos interessados. Assim, quando um livro está indisponível, o usuário entra na fila, e depois, quando o livro for devolvido e o prosseguimento for possível, o registro que está no início da fila é tratado antes dos demais.

A estrutura da fila é composta por dois ponteiros: início e fim. O ponteiro início aponta para o primeiro elemento da fila, enquanto o ponteiro fim aponta para o último. Cada elemento da fila é um nó contendo uma reserva e um ponteiro para o próximo nó. Esse formato permite inserção no final e remoção no início de maneira organizada, mantendo o comportamento FIFO sem necessidade de percorrer toda a estrutura a cada operação.

As principais operações implementadas são a criação da fila, a inserção de reservas, a remoção de reservas, a verificação de fila vazia e a exibição das reservas cadastradas. A função de criação inicializa a fila com início e fim nulos. A função de enfileiramento aloca um novo nó, armazena os dados da reserva e o adiciona ao final da estrutura. Já a remoção retira o primeiro elemento da fila e ajusta os ponteiros internos, garantindo que o próximo usuário na ordem de chegada seja atendido. A função de verificação de fila vazia retorna se não há elementos cadastrados, e a função de exibição percorre a fila mostrando todas as reservas atualmente armazenadas.

5. Descrição da Lista Encadeada

A lista encadeada foi utilizada para armazenar o histórico de empréstimos realizados no sistema. Sua função é registrar cada operação de empréstimo concluída com sucesso, guardando informações essenciais como o nome do usuário, o código do livro e o título da obra emprestada. Essa estrutura é importante porque permite manter um controle organizado das movimentações da biblioteca, possibilitando a consulta posterior dos empréstimos efetuados.

A escolha da lista encadeada é adequada para esse tipo de registro porque novos empréstimos podem ser adicionados ao histórico ao longo de toda a execução do sistema. Assim, a estrutura suporta bem inserções sucessivas e mantém os registros em sequência, refletindo a ordem em que os empréstimos foram realizados. Isso torna a lista uma solução simples e eficiente para representar o histórico da biblioteca.

As operações implementadas para essa estrutura incluem a criação da lista, a inserção de empréstimos, a listagem completa do histórico e a verificação de lista vazia. A função de criação inicializa a estrutura com o ponteiro de início nulo. A inserção adiciona um novo registro ao final da lista, preservando a ordem dos empréstimos. A função de listagem percorre todos os nós e exibe os dados armazenados em cada empréstimo. E a função para verificar se a lista está vazia retorna verdadeiro quando não há registros cadastrados.

6. Descrição das funcionalidades

- Cadastro

O cadastro permite inserir novos livros no sistema. O usuário informa o código, o título, o autor, o ano de publicação e a quantidade total de exemplares. Antes de inserir, o sistema verifica se já existe um livro com o mesmo código, evitando duplicidade. Se o código for válido e ainda não estiver cadastrado, o livro é criado e inserido na árvore binária de busca.

- Busca

A busca localiza um livro pelo código informado pelo usuário. O sistema percorre a árvore até encontrar o nó correspondente. Se o livro existir, seus dados são exibidos. Caso contrário, o sistema informa que o livro não foi encontrado.

- Listagem

A listagem dos livros pode ser feita em três percursos da árvore: em ordem, pré-ordem e pós-ordem. A listagem em ordem mostra os livros em ordem crescente de código. A pré-ordem visita primeiro o nó atual, depois a subárvore esquerda e, por fim, a direita. A pós-ordem percorre primeiro as subárvores e deixa o nó atual por último.

- Empréstimo

O empréstimo é realizado com base no nome do usuário e no código do livro. O sistema verifica se o livro existe e se ainda há exemplares disponíveis. Se houver, a quantidade disponível é reduzida e o empréstimo é registrado no histórico. Se não houver exemplares, o sistema pode encaminhar o usuário para a fila de reservas.

- Devolução

A devolução aumenta a quantidade disponível do livro informado. Depois disso, o sistema verifica se existem reservas associadas àquele código. Se houver, o próximo usuário da fila pode ser atendido, mantendo a ordem de chegada.

- Reservas

As reservas são controladas por uma fila. Quando um livro está indisponível, o usuário pode entrar nessa fila, junto com o código do livro desejado. A ordem de atendimento segue a política FIFO, ou seja, o primeiro a entrar é o primeiro a sair.

- Histórico

O histórico registra todos os empréstimos concluídos com sucesso. Cada registro guarda o nome do usuário, o código do livro e o título da obra. Esses dados são inseridos em uma lista encadeada, permitindo consultar depois quais empréstimos foram realizados no sistema.

7. Análise de complexidade

A operação de **busca na árvore binária de busca** consiste em comparar o código procurado com o código armazenado em cada nó visitado. A cada comparação, a busca continua apenas em uma das subárvores, reduzindo a quantidade de elementos analisados. Em uma árvore balanceada, essa operação possui complexidade **$O(\log n)$** .

A **inserção na árvore binária de busca** segue um processo semelhante ao da busca, pois o algoritmo percorre a árvore até encontrar a posição adequada para o novo nó. Dessa forma, sua complexidade também é **$O(\log n)$** .

O **percurso em ordem** visita todos os nós da árvore exatamente uma vez, seguindo a sequência esquerda, nó e direita. Como cada elemento é processado uma única vez durante a execução, a complexidade dessa operação é **$O(n)$** .

Na estrutura de **fila**, a **inserção** de uma reserva é realizada diretamente no final da fila por meio do ponteiro para o último elemento. Como não é necessário percorrer a estrutura para encontrar a posição de inserção, a operação possui complexidade **$O(1)$** .

De forma semelhante, a **remoção da fila** ocorre sempre no início da estrutura. Como a operação exige apenas a atualização de alguns ponteiros, sua complexidade também é **O(1)**.

A **inserção na lista encadeada** de histórico é realizada ao final da estrutura. Na implementação desenvolvida, é necessário localizar o último nó antes de efetuar a inserção, o que exige o percurso da lista. Por esse motivo, a complexidade dessa operação é **O(n)**.

A seguinte tabela apresenta um resumo das complexidades das principais operações implementadas no sistema.

Operação	Complexidade
Busca na Árvore	$O(\log n)$
Inserção na Árvore	$O(\log n)$
Percurso em ordem	$O(n)$
Inserção na Fila	$O(1)$
Remoção na Fila	$O(1)$
Inserção na Lista	$O(n)$

8. Testes realizados

Com o objetivo de verificar o funcionamento das funcionalidades implementadas, foram realizados testes envolvendo cadastro, busca, empréstimo, devolução e controle de reservas. Os principais casos de teste são apresentados a seguir.

- **Teste 1 – Cadastro de Livro**

Entrada:

Código: 101

Título: Estruturas de Dados

Autor: Mark Weiss

Ano: 2011

Quantidade total: 5

Saída esperada:

Livro inserido com sucesso

Resultado obtido:

O livro foi cadastrado corretamente e passou a fazer parte da Árvore Binária de Busca.

- **Teste 2 – Busca de Livro Existente**

Entrada:

Código: 101

Saída esperada:

Dados do livro:

Título: Estruturas de Dados

Ano de publicação: 2011

Autor: Mark Weiss

Quantidade disponível: 5

Quantidade total: 5

Resultado obtido:

O sistema localizou o livro corretamente e exibiu todas as suas informações.

- **Teste 3 – Busca de Livro Inexistente**

Entrada:

Código: 999

Saída esperada:

O livro não existe na biblioteca.

Resultado obtido:

O sistema identificou corretamente que não havia nenhum livro cadastrado com o código informado.

- **Teste 4 – Empréstimo com Livro Disponível**

Situação inicial:

Livro de código 101 com 5 exemplares disponíveis.

Entrada:

Usuário: João Lucas

Código do livro: 101

Saída esperada:

Mensagem de confirmação do empréstimo.

Resultado obtido:

O empréstimo foi registrado com sucesso, a quantidade disponível foi reduzida para 4 exemplares e um novo registro foi adicionado ao histórico de empréstimos.

- **Teste 5 – Empréstimo com Livro Indisponível**

Situação inicial:

Livro de código 102 com 0 exemplares disponíveis.

Entrada:

Usuário: Maria Souza

Código do livro: 102

Saída esperada:

Mensagem informando a indisponibilidade do livro e oferecendo a opção de reserva.

Resultado obtido:

O sistema identificou a indisponibilidade do exemplar e permitiu a inclusão do usuário na fila de reservas.

- **Teste 6 – Exibição da Fila de Reservas**

Situação inicial:

Reserva cadastrada para o livro 102.

Saída esperada:

Usuario: Maria Souza | Codigo do livro: 102

Resultado obtido:

A reserva foi exibida corretamente, demonstrando o funcionamento da fila FIFO.

- **Teste 7 – Exibição do Histórico de Empréstimos**

Situação inicial:

Empréstimo realizado pelo usuário João Silva.

Saída esperada:

Usuário: João Silva

Código do livro: 101

Título do livro: Estruturas de Dados

Resultado obtido:

O histórico apresentou corretamente os dados armazenados na lista encadeada.

9. Conclusão

O desenvolvimento deste trabalho foi uma oportunidade para colocar em prática diversos conceitos estudados na disciplina de Estrutura de Dados. Ao longo da implementação do sistema de gerenciamento de biblioteca, foi possível compreender melhor o funcionamento de estruturas como a árvore binária de busca, a fila e a lista encadeada, além de exercitar o uso de

ponteiros, alocação dinâmica de memória e a organização do código em diferentes módulos.

Um dos principais aprendizados obtidos durante o projeto foi perceber que cada estrutura de dados possui características que a tornam mais adequada para determinados problemas. A árvore binária de busca foi utilizada para organizar os livros e permitir buscas eficientes por código, enquanto a fila foi empregada para controlar as reservas respeitando a ordem de chegada dos usuários. Já a lista encadeada mostrou-se uma solução simples e adequada para armazenar o histórico de empréstimos realizados.

Durante o desenvolvimento também surgiram algumas dificuldades, principalmente relacionadas à manipulação de ponteiros e à integração entre as diferentes estruturas do sistema. Em vários momentos, o tratamento da memória dinâmica exigiu atenção para evitar erros e vazamentos de memória.

De modo geral, os objetivos propostos pelo trabalho foram alcançados. O sistema desenvolvido é capaz de cadastrar e buscar livros, realizar listagens utilizando diferentes percursos da árvore, controlar empréstimos e reservas e manter um histórico das operações realizadas. Com isso, foi possível aplicar na prática conceitos que muitas vezes são vistos apenas de forma teórica em sala de aula.

Como trabalhos futuros, o sistema pode ser expandido com funcionalidades adicionais, como a remoção de livros da árvore, a busca por título ou autor, o atendimento automático das reservas quando um livro for devolvido e melhorias na persistência dos dados. Essas alterações tornariam a aplicação mais completa e próxima de um sistema de biblioteca utilizado em situações reais.